University of Macau

Faculty of Science and Technology

Department of Computer and Information Science

澳門大學

UNIVERSIDADE DE MACAU

UNIVERSITY OF MACAU

CISC3003

Web Programming

Group Project Report

Chen Joaquin Antonio, DC226973
Tan Pak Long, DC226991
Uriel Wuli, DC227352
Seki Tomoki, DC227171
Chan Hio Tou, DC325713
Xu WuSiyuan, DC327035

20/04/2026

Table of Contents

Abstract

This project report details the development process of the web application – "The Mystery of Train Tracks", an advanced web game developed in full-stack to showcase a mastery of web development skills. This game acts as the practical combination of front-end responsive designs, back-end APIs, and secure database management.

The main gameplay involves solving a Train Track Puzzle where players have to create paths by manipulating trains in the track, created using Canvas / Web API technology for better animation. The technology stack used for developing this web application includes HTML, CSS, and JavaScript programming languages for developing a responsive web interface that will work on desktop, tablet, and mobile devices. The back-end of this application is based on PHP and Node.js, offering important functions like JWT-based authentication and session management.

The key functional features include:

- Secure Authentication: This is an advanced authentication method that uses password hashing and SMTP-based email to verify the creation of accounts.
- Search & History: The tool enables players to search for player records and puzzle solutions, along with a personal history tracker allowing players to view and delete their logs.
- Admin Supervision: This feature offers a control panel that helps track system functioning and manage aggregated player data.

The development process involved the application of Agile methodology, where work was assigned to three different groups that had expertise in designing the User Interface, back end, and database. To guarantee reliability, there was extensive testing that was performed on eight unique test cases. In addition, GitHub Actions and Heroku were utilized during deployment to develop this gaming portal. Throughout the development of the web application, team members not only learn hard skills such as using web technologies to develop amazing websites and web applications, but also a chance to enhance their soft skills like project management and team communication skills.

Context

Our project, titled "The Mystery of the Railway: A Full-Stack Interactive Online Game," addresses the core challenge of creating a responsive, secure, and feature-rich web application that simultaneously delivers an engaging gaming experience and robust user service within a puzzle environment.

In this context, our project is not merely a course assignment but a comprehensive practice of full-stack development. It fully integrates front-end responsive design, back-end API and database management, and user authentication and security mechanisms, comprehensively covering the learning outcomes of the course. Team members will collaborate, each responsible for different aspects: UI design, authentication and search functionality, database schema and logging system, and will employ agile methodologies for iterative development and testing.

The following describes how the game works, represented in UML diagrams.

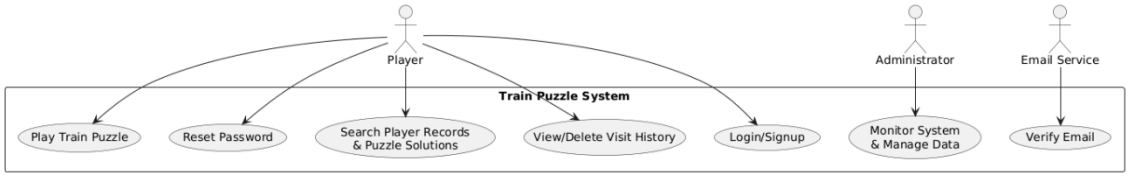

Figure 1: The web game illustrated as UML Case Diagram.

As seen in Figure 1, the main actors include “Player”, “Administrator” and “Email Service”. For each of the actors, they can perform the actions as follows:

- Player
  - Play Train Puzzle
  - Reset Password
  - Search Player Records and Puzzle Solutions
  - View and Delete Visit History
  - Login and Signup

- Administrator
  - Monitor System
  - Manage Data
- Email Service – Verify Email

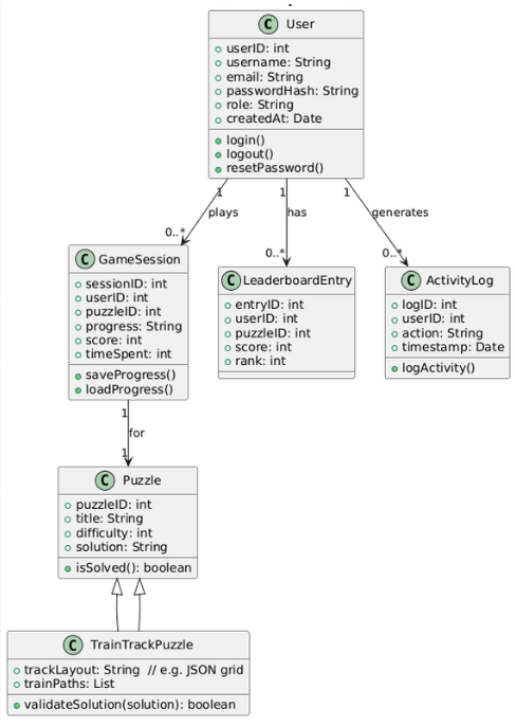

Figure 2: The web game illustrated as UML Class Diagram.

As illustrated in Figure 2, the classes included in the web game consist of:

- User (User Class): Stores user information and provides login, logout, and password reset functions.
- GameSession (Game Session Class): Records the player's progress, score, and time spent on puzzles.
- Puzzle (Puzzle Class): Defines the puzzle's title, difficulty, and solution method.
- TrainTrackPuzzle (Train Track Puzzle Class): Inherits from Puzzle, adds track layout and train paths, and can verify solutions.
- LeaderboardEntry (Leaderboard Entry Class): Saves player scores and rankings.
- ActivityLog (Activity Log Class): Records user actions and timestamps.

The BPMN workflow illustrated in Figure 3 consists of five main components:

- User Registration/Login
  - First, the user submits credentials to the system for validation. Then, it triggers email verification. Thus, the user's account is activated.
- Gameplay Flow
  - The user starts playing the train puzzle game. The game logic executes as the user plays along. After the game has ended (e.g. the train crashed, the user passed all levels successfully), the results are stored in the database and updated.
- Search Service

- The workflow starts by a user entering a query, then the API fetches results from the database, and the results are displayed in the website's User Interface (UI).

- Visit History Tracking
  - For each page visit logged, it is then stored in the history table such that users can either view or delete their visiting history.
- Admin Oversight
  - The administrator first accesses the dashboard, then he/she can check aggregated player data. This way, audit logs can be maintained.

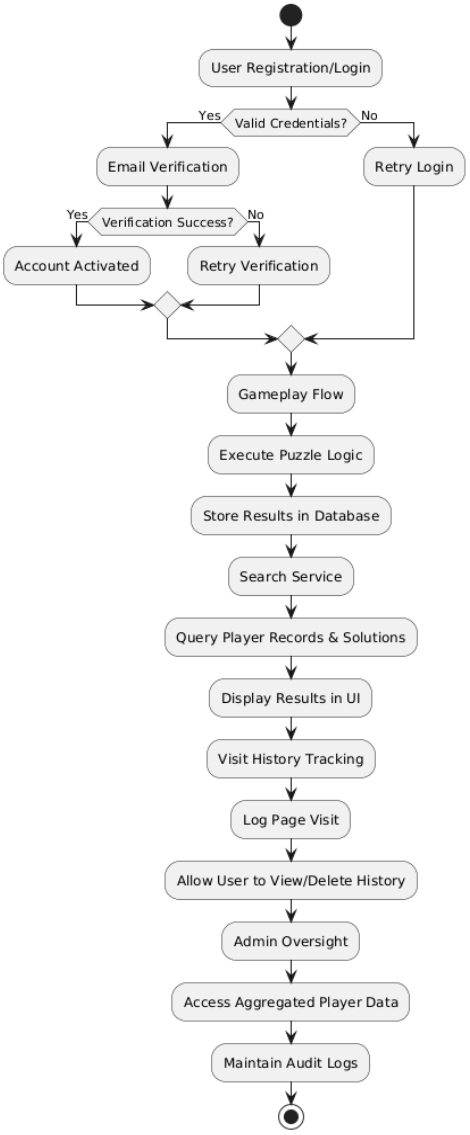

Figure 3: The web game illustrated in BPMN workflow.

The overview of the web game's architecture is shown in Figure 4 below. To elaborate,

- The frontend: The user interface of the application which allows the player to play the game through their web browser. It includes typical technologies like HTML, CSS, and JavaScript. There is an element dedicated specifically to the Puzzle Game Canvas (Grid). As per the provided diagram, the canvas or WebGL is used to draw smooth lines of train tracks. In addition, this UI adjusts to different layouts according to whether the game is played on a computer, tablet or smartphone.
- The communication layer: The Frontend communicates with the Backend using HTTPS requests, interacting with the RESTful API that allows for such processes as saving the player's progress, authenticating players and fetching puzzle-related information.
- The backend: This is the core of the web game. With the help of Node.js and

PHP technologies, it processes the following use cases – the ones mentioned in the previous paragraph: "Signup/Login" and "Reset password". This is done through implementing JWT or session authentication and securing user sessions. It also performs checks of puzzles in order to ensure that there is a possibility of a train getting from point A to point B on the map, and serves the “Search Player Records” use case.

- The database: The storage type for the web game would be MySQL. It manages specific tables for Users, Puzzles, Sessions (to keep track of who is logged in), and Logs, which likely stores that "Visit History" the players want to see or delete.

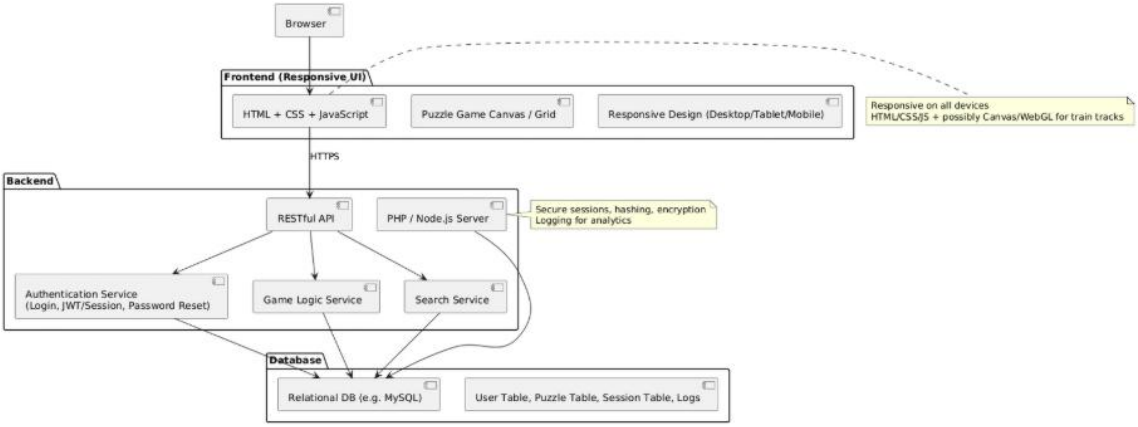

Figure 4: The overview of the web application in front-end and back-end perspectives.

Goals

To achieve the completeness of the web application, we split our project goals into five main aspects: front-end, back-end, database, security and deployment.

Firstly, for the website’s front-end, the web application must be responsive and adaptable with various devices. The game should be easy to play with a simple yet decorative interface (e.g. tracks, trains and puzzle logic), as well as menus and player result display.

Secondly, the website’s back-end should be built with an API service, as well as the

account login and registration system with email verifications and password reset. The system should also manage sessions such that only registered players can access the home page. Additionally, a search interface is provided for players to look for their records and puzzle solutions.

Thirdly, regarding databases, the most fundamental schemas such as users, game history and access logs must be defined properly. As more and more users enter and play the game, we must implement data storage, querying and log tracing to accurately store and handle each player's data. We also focus on optimizing query performance using indexes and caching.

Regarding the security of the web application, hash algorithms must be applied to protect users' passwords, along with implementing encrypted communication and secure sessions. The game should also provide user privacy settings like deleting browsing history. Once the testing is complete without any errors and bugs, we must deploy our product smoothly by leveraging GitHub automated workflows, ensuring the website is accessible online and supports multi-user concurrency.

Services and Data Requirements

The service of the web application is the train puzzle game itself, including puzzle logic, scoring function, and difficulty progression and customization. The game also pairs with the functionalities described as follows:

- Authentication: Besides playing the game, users can create their own accounts with email verification, and they can reset their password when forgotten.
- Search: Users can search for records and data from other players, as well as puzzle solutions in case they are stuck to a certain level.
- Visiting History: Consists of visit logs with timestamps.

The following lists the data required for the web application:

- User Table: The basic data of a user include accounts, credentials and verification status.
- Game Records: A table to store players' scores and the number of attempts.
- History Table: A table to save users' visiting history and timestamps.
- Puzzle Solutions: A table to store the solutions of the puzzle.

Tasks and Tools

To ensure our project is done smoothly, we split the tasks into three major procedures: system design, implementation, and testing and deployment. First, we outline the system architectures, create User Interface (UI) wireframes and database ER diagrams. Once the outline and proposal are unanimously agreed by us, we start developing the game by building responsive UI, developing authentication and search APIs and designing database schemas. We start by working each aspect separately, and gradually combining all together for unit and usability testing. When the game is ready with all functionalities running successfully, we deploy it to a PHP-enabled cloud platform.

The tools required for this project are listed as follows:

- Front-end: The core technologies such as HTML, CSS and JavaScript, with HTML for content markups, CSS for attractive and responsive design, and JavaScript for the main game loop and logic.
- Back-end: PHP is applied because of it integrating seamlessly with MySQL and phpMyAdmin, making it suitable for implementing core functionalities like user registration, login and account activation. Node.js is also used for asynchronous non-blocking I/O, which meets the need of high-concurrency requests and facilitating future expansion into more complex API services or real-time interactive functions.
- Database: MySQL would be the main tool for working out the database system. Advantages include strict data structure, complex analytical queries, and absolute transactional reliability.
- Deployment: Not only do we deploy our final product to GitHub, but we also

deploy it to Heroku, which supports full-stack web deployment.

- Other: GitHub is crucial for deploying our web application. For instance, GitHub Actions provides a consistent, reproducible environment for deployments, reducing "it works on my machine" errors.

Challenges

We expect challenges such as integrating the game logic into the full-stack web framework smoothly, ensuring secure user authentication, and managing real-time history tracking with no performance issues. As we progress on developing the web application, we face several challenges described below:

1. How can we integrate the game logic into the full-stack web framework smoothly?
2. How can we ensure the application includes secure user authentication? Are methods such as password hashing and email verification applied during the verification stage? How can we avoid common vulnerabilities such as SQL injection and XSS attacks?
3. How can we achieve cross-device responsiveness?
4. How can we manage real-time history tracking without performance issues?
5. How can we design efficient search queries for players' records and puzzle solutions?

What is more, we encounter questions from different teams, making us think further when we are developing the game.

Since we start with working each aspect by separate pairs, an agile approach will be adopted where there will be a well-defined distribution of duties among all members. The front-end developers will concentrate on creating user-friendly UI components, whereas the back-end programmers will work towards designing APIs that provide well-organized endpoints. The database programmers will work towards ensuring

consistency in schemas and optimizing the queries involved. Continuous sprint meetings and integration testing will help in spotting mismatches between different system tiers.

In a certain case if our system scales up, we will employ a layered architecture and load balancing to distribute request pressure, while using indexes, caching, and partitioning at the database level to maintain query performance. The frontend enhances user experience through reactive design and asynchronous requests, while the backend can be scaled to a microservice architecture as needed to ensure performance and responsiveness under high concurrency.

As the number of users grows, we will employ efficient indexing mechanisms, caching strategies, and, when necessary, separate player data from personal profiles. At the same time, users will have data autonomy, able to delete their browsing history in their personal settings to ensure privacy.

Deliverables

The following is a list of deliverables for the web project.

- Fully functional Train Puzzle Game
- Secure login/signup system with email verification
- Search service for records and solutions
- Visiting history and tracking
- Documentation (e.g. README, system design, user guide)
- Demo gameplay

Division of Work among Team Members

We split this web application into three different aspects: front-end interactive design, back-end development and database management. The following are the distributions of the tasks.

- Pair 20 (Bruce and Collins) – Front-end Interactive Design

Pair 20 is responsible for building a responsive UI for the puzzle game, including main game loop, menu and players' results.

The train puzzle board is the primary component of the game. Therefore, they start by building the core gameplay prototype, focusing on rendering blocks (e.g. black blank space, rotatable train tracks and red trap) and the train's visuals. Then, they gradually work on train movements, track interactions and difficulty settings. After that, they work on a simple level by creating a 7x7 train puzzle at the center of the website, for implementing game logic and algorithms. Once confirmed the smooth performance and controls of the simple level, they extend the game by adding more levels, as well as creating features of "Menu" and "Results". The game will then be attractive and accessible for users with various devices.

- Pair 11 (Uriel and John) - Back-end Development

Pair 11 is in charge of building a comprehensive and responsible website back-end. Their work is divided into a few sections: user registration, authentication, account recovery and data storage. The core infrastructure includes:

- Database (PHP and MySQL) – Singleton connection and user table as core;
- Mailer (PHPMailer) – Applies SMTP-based email, used for activation and password reset
- Dependencies (JSON / Vendor) – To manage external libraries and support email security
- Global Styling (CSS) – Utilizes unified brutalist UI, ensures consistent design

A simple login/signup user interface is made with similar styles of the web game to ensure seamless and responsive user experience. Inside, the signup page using HTML

and JavaScript collects user data and implements real-time validation. During the signup process, PHP is applied to store user status as “inactive”, then to send activation email along with secure token generated. Users will be notified and prompted to check their email by simple HTML markup. Finally, the back-end validates whether the token is still valid or not. If so, the user’s account is then activated successfully.

However, creating a user account for the game is optional. Therefore, login and access control must be done to separate registered players and guests. Users with activated accounts exclusively are granted access to the homepage. They can also logout their accounts, so the back-end is obliged to destroy the session to ensure security. For guests, they are entered into “Visitor Mode”, where they can play without login and with data not being saved into the website’s database.

Regarding account recovery, a “Forget Password?” section is made for users who forget their password during login. With PHP aided, the system generates a time-limited token and sends an email for password reset. Once confirmed the email and token are valid, users are then prompted to enter a new password in the reset page, and the new data will be passed to the database system for storage.

- Pair 15 (Seki and Antonio) - Database Management

Pair 15’s task is to construct the core database system for the web application, including database schemas, data operations and audit logging.

First, a schema is built with core entities such as “User”, “PlayerProfile”, “GameRecord” and. “VisitLogs”. Then, using MySQL queries, they perform game data operations to save records, update scores, retrieve profiles and query game history. An activity logging system is made to track users’ visiting history, which retrieves login IP, device information and timestamps. Finally, the database supports the back-end with user authentication and data queries, ensuring secure and efficient access to player and game data.

They also aid Pair 11 to achieve a secure login workflow for the web game. To illustrate

the workflow, the player enters credentials in the game UI, which sends a login request to the PHP backend. Then, the backend retrieves the user's account data, including password hash and verification status, from MySQL. Here, a two-step validation is implemented where the system checks if the email is verified, then verifies the password using secure hashing. Successful logins are recorded in visit_logs, and a session token is issued to grant the player access to the game.

Contributions from Individual Team Members

Here are work contributions from each team member:

- Collins (from Pair 20): Preliminary idea integration, project management, writing of report and presentation, web application testing.
- Bruce (from Pair 20): Co-project management; main game development (e.g. the core gameplay logic, level selection, endless mode, scoring and ranking system), front-end and back-end design, integration of the game-related features.
- Uriel (from Pair 11): Design search system and email functionality for user feedback.
- John (from Pair 11): Design the login web page and implement the login interface functionality.
- Seki (from Pair 15): Database architecture and management (designing relational schemas for users, puzzles, and gameplay records ensuring data integrity); executed cloud server deployment and environment configuration (via Alibaba Cloud); led the full-stack integration, which included implementing cross-page session management (PHP), resolving API routing conflicts, fixing legacy local-path bugs in the email system for cloud deployment, and optimizing complex SQL queries for accurate Global Leaderboard rendering. Also engineered dynamic mock data seeding for project demonstration.
- Antonio (from Pair 15): Co-responsible for database management and backend API implementation (e.g., developing and refining data endpoints like save_score.php and get_leaderboard.php); assisted in full-stack integration and system debugging, ensuring seamless data communication between the frontend

UI and backend database; conducted comprehensive system testing to verify game state saving, leaderboard accuracy, and overall deployment stability.

Criteria for Success

By following the criteria below, we are confident that the project will result in a game meeting our requirements:

- Responsive design & UI: Is the web game playable on devices like desktop, tablet and smartphone? Is the UI intuitive such that it ensures smooth gameplay?
- Search function: Does the search system work properly with players' records and puzzle solutions available? Are users' visiting history logged clearly?
- Secure authentication: Are session management handled properly with separation between registered users and guests? Are users' accounts and their corresponding password stored securely with hash functions?
- Documentation & Deployment: Does the final product contain documentations such as user guide and game information? Is the game deployed successfully without any errors?

After careful checks on the above criteria, the final product should possess the following characteristics:

- The website loads completely without JavaScript or CSS errors.
- The web must consist of responsive design, ensuring the game displays well on any devices.
- The UI provides smooth gameplay with intuitive controls.
- The application includes a secure login/signup system with email verification and password reset.
- The webpage offers search functionality for player records and puzzle solutions.
- The web game tracks visiting history with clear logs accessible to users.
- Outside of the website, there should be documentation such as README, annotated examples and, if possible, a demo gameplay.

- The project is successfully deployed on a PHP-enabled platform.

The scoring criteria are as follows:

- Content accuracy and relevance (35%)
- Technical implementation and functionality (35%)
- Design and user experience (20%)
- Documentation completeness (10%)

Evaluation

We set up an assessment rubric to keep track of our workflow and completeness. The following table displays how we evaluate each aspect of the web application.

Criteria	**Excellent (90-100%)**	**Good (75-90%)**	**Fair (60-74%)**	**Needs Improvement (<60%)**
Content Accuracy and Relevance	The project matches the proposal; all features (e.g. gameplay, authentication, search, etc.) are implemented correctly and in line with the course objectives.	Most features are implemented seamlessly with minor inaccuracies; overall relevant.	Some features are missing or partially implemented; not quite related to course intended learning outcomes.	Major gaps in features; project reflect neither the proposal nor the intended learning outcomes.

Technical Implementation and Functionality	No bugs exist; secure authentication works great; the site is responsive; the search service and history tracker both works great.	Workable primary functionalities with minor bugs; security and responsiveness are mostly achieved.	Inconsistent functionality; noticeable bugs; limited security or any other optimization.	Unstable system; important features not workable or missing; insecure or non-functional implementation.
Design and User Experience	The interface is clean, attractive, responsive across devices, accessibility criteria met; gameplay is smooth and enjoyable.	The user interface is clear and easy to use, with responsive design mostly achieved; minor accessibility issues.	The interface is somewhat unclear and confusing with limited responsiveness; the gameplay is not fully intuitive.	Lack of attractive design; no responsive layout; the user interface is complicated for users.
Documentation Completeness	Comprehensive documentation including system design, README file, ER diagrams, instructions, as well as user guide and demo visuals.	Documentation covers major aspects; a small part of the diagrams or user guide explained unclearly.	Incomplete documentation with key elements missing (e.g. ER diagram, deployment guide).	Very little or no documentation; hard to understand or reproduce the project.

This rubric is not only for ourselves, but it also provides instructors and teaching assistants structured grading criteria aligned with project requirements. While working on the project, each of our team members will ensure each “Excellent” descriptor is satisfied, and will share our grading and thoughts to the team for further improvements.

Project Learning – Individual and Team

Throughout the development of the game, we not only learned hard skills such as using web technologies to create a web application, but also developed our teamwork and communication abilities, which are essential for our future careers.

As for the individual skills of our team members, working on the web game according to the skills acquired from the course made us realize that we significantly improved our ability to develop a full-stack application. In addition, the division of the work for this task gave us an opportunity to expand our expertise even further than what was discussed in the course. For instance, through the front-end development part of the process, we gained more knowledge about designing interfaces for various types of screens. In turn, through the back-end development, we learned more about additional website functionality, including authentication, data storage for players, and puzzle solutions. Finally, managing the database expanded our database-related knowledge received during the introduction course offered by UM.

Working as a team, we were able to develop soft skills that increased not only our efficiency at working but also the quality of the work done. For example, we used agile development techniques to allow us to make quick progress as well as make adjustments along the way. In addition to this, we were able to improve on our project management skills and become more efficient at communicating with each other. Not only did this teamwork experience teach us about how to collaborate on projects from a technical perspective, but also on how to empathize and be adaptable in a professional setting.

Functional Requirements and Test Cases

Regarding the use cases described in the project context, the following displays the test cases for functional requirements granted to each actor.

1. Player

FR1: Authentication (Login / Signup)

- FR1.1: The system shall allow users to create an account using a unique username, email, and password.
- FR1.2: The system shall authenticate users via a secure login portal.

Test Case ID	TC-01
Description	Successful Signup
Prerequisites	The user is on the Signup page.
Test Steps	1. Enter valid email, unique username, and password. 2. Click “Sign Up”.
Expected Results	Account created; user redirected to the dashboard.

Test Case ID	TC-02
Description	Invalid Login
Prerequisites	The user’s account exists.
Test Steps	1. Enter the correct username but wrong password. 2. Click “Log In”.
Expected Results	Error message: "Invalid credentials.

FR2: Reset Password

- FR2.1: The system shall allow users to request a password reset link via their registered email.
- FR2.2: The system shall update the password in the database upon successful verification.

Test Case ID	TC-03
Description	Password Reset Flow
Prerequisites	The user has forgotten the password.
Test Steps	1. Click “Forgot Password?”. 2. Enter email. 3. Click the link in the email. 4. Enter new password.
Expected Results	Password is updated; the user can login with the new password.

FR3: Play Train Puzzle

- FR3.1: The system shall load puzzle levels with specific "train track" logic constraints.
- FR3.2: The system shall track the time taken and moves made to complete a puzzle.

Test Case ID	TC-04
Description	Puzzle Completion
Prerequisites	Level 1 is loaded.
Test Steps	As the train approaches, arrange tracks to connect from one point to another.
Expected Results	Successful animation triggers; “Level Complete” record saved.

FR4: Search Records & Solutions

- FR4.1: Users shall be able to search for other players' public profiles and high scores.
- FR4.2: Users shall be able to view shared solutions for completed puzzles.

Test Case ID	TC-05
Description	Search Funtionality
Prerequisites	Records exist in the database.
Test Steps	1. Go to “Leaderboard”. 2. Type a username in the search bar.
Expected Results	Search results display the specific player’s profile.

FR5: Visit History (View / Delete)

- FR5.1: The system shall log each time a player visits a puzzle level.
- FR5.2: Players shall be able to view a list of their recently visited puzzles and delete specific entries.

Test Case ID	TC-06
Description	Delete History
Prerequisites	History contains 5 entries.
Test Steps	1. Go to “My History”. 2. Click “Delete” on the top entry.
Expected Results	Entry is removed; list refreshes to show 4 entries.

2. Administrator

FR6: Monitor System & Manage Data

- FR6.1: Administrators shall have a dashboard to view system health (uptime,

active users).

- FR6.2: Administrators shall have the authority to delete inappropriate puzzle solutions or user-generated data.

Test Case ID	TC-07
Description	Admin Data Deletion
Prerequisites	Logged in as Admin.
Test Steps	1. Navigate to “Manage Data”. 2. Select a flagged solution. 3. Click “Remove”.
Expected Results	The solution is purged from the database and public view.

3. Email Service

FR7: Verify Emails

- FR7.1: The system shall interface with an external SMTP/Email Service to send verification tokens.
- FR7.2: Accounts shall remain "Pending" until the email verification link is clicked.

Test Case ID	TC-08
Description	Email Verification
Prerequisites	The user just signed up.
Test Steps	1. Check inbox for verification email. 2. Click the unique token link.
Expected Results	Status changes from “Pending” to “Verified” in the database.

Demo Walkthrough

This section is a guide of how the web game works, with screens shots and description to aid our explanations.

Step 1: Create a 3D effect map and determine the types of railways, as well as the overall color tone of the scene.

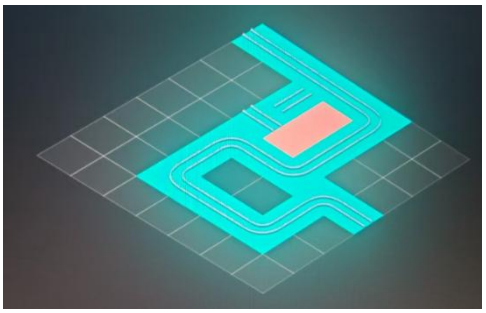

Figure 5: The train puzzle with navigable railway route.

The logical rule is set as follows:

1. Generate a navigable railway route;
2. Generate the remaining land parcel by random, with overall effects consistent with that shown in Figure 5.

Step 2: Design the train model and railway alignment logic, ensuring that the train can move along the tracks and stop at any broken track sections. (as seen in Figure 6)

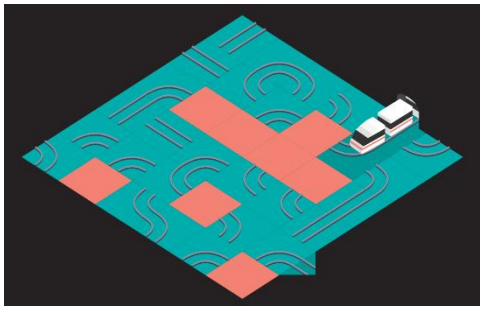

Figure 6: The train puzzle with the rest of the tiles filled up. The railway is now unstructured.

Step 3: Design ten game levels, launch an endless mode, and develop a unique scoring algorithm to enrich the diversity and playability of the game. Meanwhile, the initial version of the home page is also designed. (as seen in Figure 7)

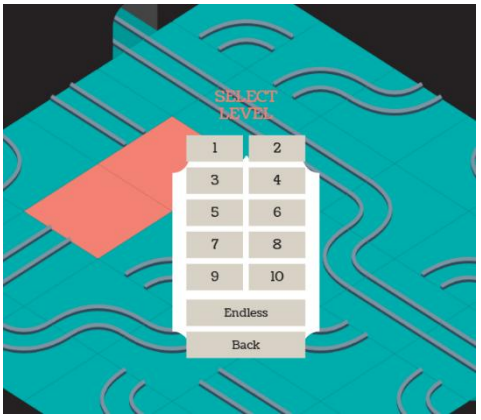

Figure 7: A menu consisting of level and game mode selection.

Step 4: Eliminate logical infinite loops in the program, conduct game testing, and start handover work with team members responsible for the database and login interface. (as seen in Figure 8)

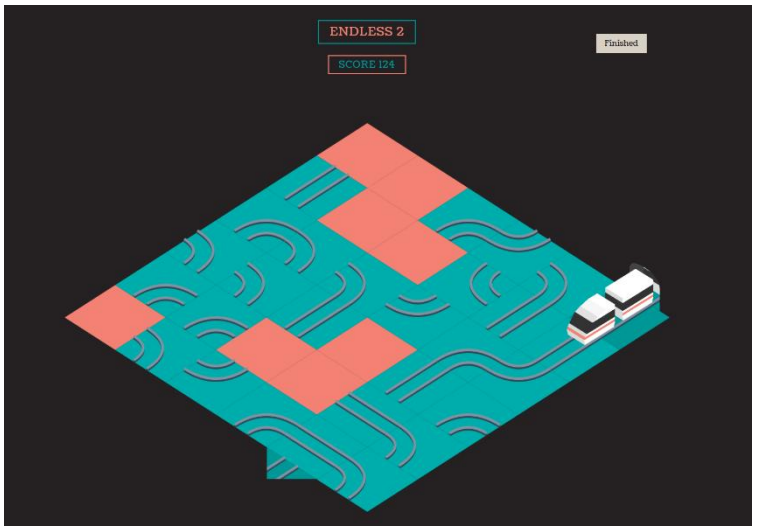

Figure 8: The train puzzle game in endless mode.

References

Tirholm, J. (n.d.). Train Puzzle Game Prototype. CodePen. Retrieved from https://codepen.io/johan-tirholm/pen/xoNPzN

Freeman, E., Robson, E., Bates, B., & Sierra, K. (2004). Head First Design Patterns. O'Reilly Media.

Mozilla Developer Network (MDN). (n.d.). Web Security. Retrieved from https://developer.mozilla.org/en-US/docs/Web/Security

W3C. (2018). Web Content Accessibility Guidelines (WCAG) 2.1. Retrieved from https://www.w3.org/TR/WCAG21/

Connolly, T., & Begg, C. (2014). Database Systems: A Practical Approach to Design, Implementation, and Management. Pearson.

Declaration of AI Usage

We declare that our web application is made with the help of the following AI models:

- Gemini 3.1 Pro – Used for code debugging, database SQL optimization, and generating mock data.
- Github Copilot – Provide assistance with HTML, CSS, JS, PHP and SQL code writing.